

Part 5

Scripting in TF3: transformators, and what carries over from TSC Lua

TF3 vehicles have no per-vehicle gameplay script. What they have is a rendering-side hook, the transformator, that turns simulation state into animation states and particle settings. This part explains it, maps the TSC Lua features onto it, and is honest about what does not translate.

In this part

1. Where code can run
2. The transformator: `.trf.lua` and its four functions
3. What `updateFn` sees and can do
4. Particles
5. Emitting models
6. The TSC features, one by one
7. Case: trolley poles in TF3
8. Mod parameters and game scripts (outside this guide)
9. Pitfalls

Function names, parameters and limits are quoted from the Vehicle Advanced Topics page (“Transformator Scripts”). The example transformator download mentioned there was not supplied; shapes below follow the manual's snippets.

1 · Where code can run

Place	Runs when	Can do	TSC analogue
Transformator (<code>.trf.lua</code> + <code>.script</code>)	Every rendered frame for a vehicle in view manual	Set node transforms, start/scrub animations, tune particle emitters, emit extra models. Read-only simulation state. No api beyond <code>api.type</code> . manual	The animation/visual half of the engine script
Mod script (<code>mod.lua</code> , parameters)	Load time	Alter resources, add parameters the player sets in the mod menu (page not supplied check)	-
Game scripts	During play (page “coming soon”)	Gameplay logic, GUI, commands via <code>api.*</code>	Scenario scripting, not vehicle scripting

The one rule that shapes everything

In TSC the script *decides* (it writes the controls the simulation reads). In TF3 the simulation decides and the transformator only *draws*. Nothing a transformator does changes speed, power, loading or routing. Anything from the TSC design that alters gameplay (thermal damage reducing tractive effort, a dewired pole cutting power, brake fade) has **no TF3 counterpart** at the vehicle level.

2 · The transformator: `.trf.lua` and its four functions

```
-- myloco.trf.lua
function data() return {
  updateScript = {
    fileName = "myloco.script@myloco.updateFn", params = {} },
  updateParticleSystemScript = {
    fileName = "myloco.script@myloco.updateParticleSystemFn", params = { smokeScale = "1.5" } },
  getEmittableModelsScript = {
    fileName = "myloco.script@myloco.getEmittableModelsFn", params = {} },
  computeEmittedModelsScript = {
    fileName = "myloco.script@myloco.computeEmittedModelsFn", params = {} },
} end
```

[manual](#) Each entry names a function inside a `.script` file (`file@table.function`) and may pass a key/value `params` table that arrives as `captureParams`. The vehicle's `.mdl` references the transformator through `transformatorConfig`; without one, the **default** rail transformator runs and provides every event listed in Part 3. “Less complex transformators result in better performance.”

[manual](#)

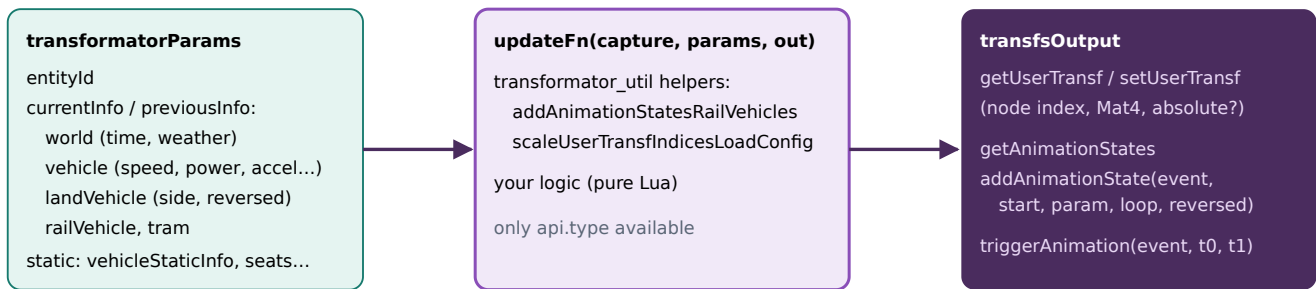


Figure 5.1 • The update function manual. Read state on the left, write animation states and transforms on the right. The manual's aircraft example scrubs a “nose_up” animation to a time derived from flight state, and triggers a transition when the state changes.

3 • What updateFn sees and can do

```

local transformator_util = require "transformator_util" -- helper library (name per manual)
local myloco = {}

myloco.updateFn = function(captureParams, params, transfsOutput)
  if params.currentInfo.landVehicle ~= nil then
    -- 1. keep the default rail behaviour (doors, parts on/off, wheels ...)
    transformator_util.addAnimationStatesRailVehicles(
      params.currentInfo.landVehicle.side, params.currentInfo.landVehicle.reversed,
      transfsOutput)

    transformator_util.scaleUserTransfIndicesLoadConfig(params.currentInfo.vehicle.indicesLoadConfig,
      transfsOutput)

    -- 2. your addition: scrub a 0..1000 ms "panto_raise" strip with speed (stopped = down,
    moving = up)
    local v = params.currentInfo.vehicle.speed or 0
    local t = math.min(1000, math.floor(math.abs(v) * 400))
    transfsOutput:addAnimationState("panto_raise", -1, t, false, false)
  end
end
return myloco
  
```

manual for the helper names, `addAnimationState(event, start, param, loop, reversed)` and `triggerAnimation(event, t0, t1)`; check the exact `require` path, the `speed` field name inside `vehicle`, and the time unit of `param` (the manual's example uses milliseconds, 11000 for an 11-second strip). Always test for the presence of `currentInfo.*` sub-tables: “it is important to check if it exists before using the content.” manual

4 · Particles

```
myloco.updateParticleSystemFn = function(captureParams, params, particleSystem)
  if params.currentInfo.vehicle == nil then return end
  local scale = 0.2 + 0.8 * (params.currentInfo.vehicle.powerOutput or 0)  -- field name: check
  for i = 0, particleSystem:getSize() - 1 do
    if particleSystem:getParticleId(i) == "emitter_smoke_dark" then
      particleSystem:setLifeTimeScale(i, scale * tonumber(captureParams.smokeScale or 1))
    end
  end
end
```

manual for `getSize`, `getParticleId`, `setLifeTimeScale` and the recommended order (check state → precompute → iterate). This is the TF3 version of TSC's `Call("Main Smoke Stack:SetEmitterRate", ...)`.

5 · Emitting models

`getEmittableModelsFn` returns the list of model paths that may be emitted (fully qualified: `"mymod::/asset/x.mdl"`); `computeEmittedModelsFn` then calls `modelEmitter:emitModel(modelId, Mat4f, animationStates, parentNode, alignToTerrain)` to spawn them relative to a node. **manual** Use it for things TSC did with child entities: a visible crew member not covered by seats, a swappable load, a trailing banner.

6 · The TSC features, one by one

TSC Lua feature (guide/ Part 5)	TF3?	How, or why not
Pass-through <code>OnControlValueChange</code>	n/a	No controls, no player input on vehicles.
Read speed, curvature, gradient	partly	<code>currentInfo.vehicle</code> gives speed/power/acceleration; no curvature or gradient field is listed check .
Write a control (virtual throttle)	no	Simulation state is read-only in transformers.
Play / scrub an animation	yes	<code>addAnimationState</code> / <code>triggerAnimation</code> .
Show / hide a node (<code>ActivateNode</code>)	yes	Default parts on/off events, or a 1-key "on"/"off" animation you trigger.
Emitter rate / colour	yes	<code>updateParticleSystemFn</code> setters.
Lights on/off, colour	partly	Lights are metadata with fixed properties; switch the mesh/light's parent node via parts events check whether light sources follow hidden nodes.
Alerts / messages	no	No UI access from transformers.
Consist messages	partly	<code>currentInfo.vehicle</code> includes "info about the configuration of the whole consist" manual ; the position events cover the common cases.

TSC Lua feature (guide/ Part 5)	TF3?	How, or why not
Thermal damage, brake fade, flat spots	no (gameplay)	Only a visual could be faked (smoke, sparks via emitters). TF3 has its own maintenance/aging model (the Age slider in the Model Editor drives dirt and rust).
Voltage zones	no	Electrification is a track property; vehicles declare ELECTRIC_TRAIN and the game enforces it.

7 • Case: trolley poles in TF3

TSC (guide/ Part 5): gameplay

hoist / seat / fall state machine
 risk roll from curvature × speed
 dewire cuts PantographControl → no traction
 route wire-height presets, keys [] P Alt+P
 alerts, sound pulses

TF3: visual only

pole_front / pole_rear instances, raised + stowed
 trailing pole up: back_forward_parts_on (front pole: _backward_)
 optional transformer: scrub "pole_raise" with speed
 wire height: one pose per route is impossible → pick one
 no dewire, no power cut, no keys

Figure 5.2 • The same model, two different jobs. In TF3 the pole is a cosmetic that must always look right, so default to the robust mesh-swap and reserve the transformer for a smooth raise in depots.

What does carry across is the **model**: the TSC `TrolleyPole.lua` assumes frames 0–100 on `poleFront / poleRear`; the same two actions, pushed to strips, are the TF3 `pole_raise` animations. The TSC design's 65/72/78/92 wire-height frames become fixed “raised” poses per route mod.

8 • Mod parameters and game scripts

Mod-level Lua (`mod.lua`, parameters shown in the mod menu) and game scripts (gameplay, GUI, missions) exist in TF3 but their pages were not supplied and are not vehicle-specific. The API index lists `api.cmd`, `api.engine`, `api.gui`, `api.res`, `api.type`; the manual recommends the in-game console and `debugPrint(table)` for exploring. [manual](#) If you want the Texas Electric gameplay (dewires, thermal damage) in TF3, it would have to be a game script acting on the whole simulation, which is a different project.

9 • Pitfalls

Pitfall	Fix
Calling <code>api.res / api.engine</code> inside a transformer	Not allowed; only <code>api.type</code> . Use <code>modelEmitter:getModelId</code> for model ids. manual
Assuming <code>currentInfo.vehicle</code> exists	It may not (model used statically); check for nil. manual
Dropping the default behaviour	A custom <code>updateFn</code> replaces the default; call the <code>transformator_util</code> helpers first or doors stop working. manual

Pitfall	Fix
Emitting a model not returned by <code>getEmittableModelsFn</code>	Not possible; list it there first. manual
Heavy per-frame work	Runs for every vehicle in view; precompute, keep it small. manual
Expecting gameplay effects	Transformators draw; they do not drive.